Introduction to Recommender Systems

Session 1 | 09:00–09:45 | Theory

ACM Student Chapter Workshop

Day 1 roadmap: one continuous story

Time	Session	You leave with
09:00	**Introduction to recommender systems**	the big picture
09:45	Lab 1: ratings data, preprocessing, EDA	an EDA notebook
11:00	Collaborative filtering (theory)	user/item similarity
11:45	Lab 2: build a CF recommender	Top-N for a real user
13:30	Matrix factorization: SVD / ALS (theory)	$R \approx PQ^\top$
14:15	Lab 3: SVD with Surprise	a working, evaluated recommender
15:45	Wrap-up and assessment	the whole pipeline



Recommendations are everywhere

Service	What is recommended	Typical signal
Netflix	titles, rows, artwork	watch history, ratings, time of day
Amazon / Flipkart	products, "bought together"	purchases, views, cart
YouTube	next video	watch time, likes, searches
Spotify	songs, playlists, Discover Weekly	plays, skips, saves
Instagram	posts, reels, accounts	likes, dwell time, follows
LinkedIn / news	jobs, people, articles	clicks, applications, reads

Ask the room (60 s)

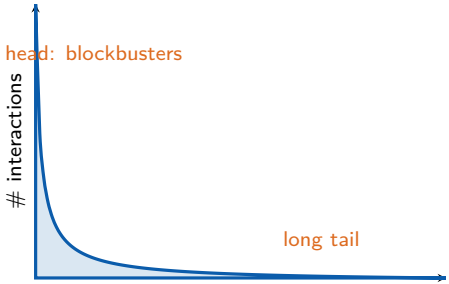
"How does Netflix decide which movie to show *you*?" Which data would you use?

Why do we need them? Information overload

- Catalogues are huge: millions of songs, products, videos
- A user can inspect only a few dozen items per session
- Search needs a query; users often *do not know what they want*
- Value lives in the long tail: niche items that a personalised list can surface

Goal

Show each user a *short, personalised list* that is relevant, useful and ideally a little surprising.



Schematic. In Lab 1 you will plot the real curve for MovieLens.

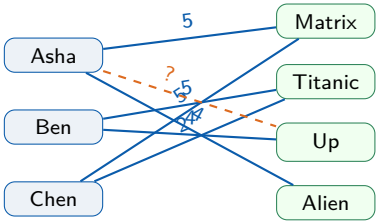
The recommendation problem

Given

- a set of users $\mathcal{U} = \{1, \ldots, m\}$
- a set of items $\mathcal{I} = \{1, \ldots, n\}$
- observed interactions r_{ui} for some pairs $(u, i) \in \Omega$
- (optional) user and item *side information*

Find for a user u the items with the highest *relevance*

$$\hat{r}_{ui} \;=\; f(u, i) \qquad i \notin \Omega_u$$



Observed ratings (solid) and a prediction we want (dashed).

In words

Predict **which item is most relevant to a particular user** from users, items and past interactions.

Two ways to pose the task

1. Rating prediction

Estimate the value of each cell, $\hat{r}_{ui}$.

"Will Asha give Up 4 or 2 stars?"

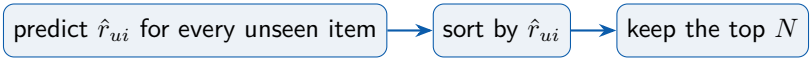
Metrics: RMSE, MAE

2. Top-N recommendation

Return an *ordered list* of N unseen items.

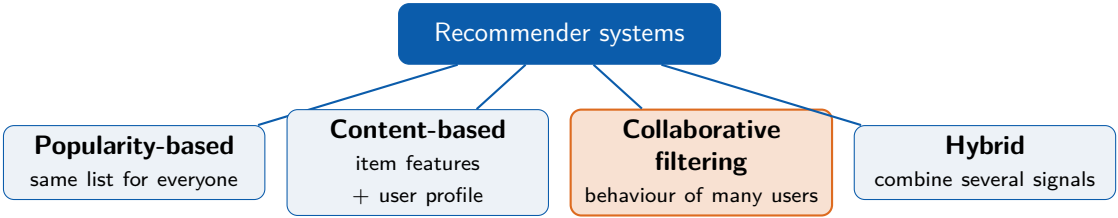
"Which 10 titles do we show Asha?"

Metrics: Precision@N, Recall@N, NDCG



But: a model with a low RMSE is not automatically good at ranking (you will see this in Lab 3).

Map of the territory



Day 1 focus: collaborative filtering (similarity-based and matrix factorization)

Type 1: Popularity-based

Idea. Recommend what is popular (or highly rated) overall.

- "Top 10 movies this week"
- Score: number of interactions, or a damped mean

$$\text{score}_i = \frac{n_i\,\bar{r}_i + m\,C}{n_i + m}$$

(n_i ratings of item i, $\bar{r}_i$ its mean, C the global mean, m prior strength)

Pros

Trivial, robust, works for *new users*

Cons

No personalisation; rich-get-richer feedback loop

Role in this course

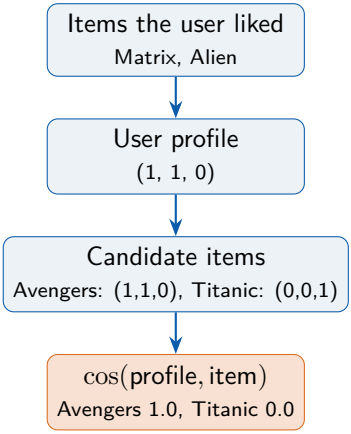
The **baseline** every model must beat (and, surprisingly, a hard one to beat for Top-N).

Type 2: Content-based

Idea. Recommend items *similar in content* to those the user liked.

1. describe items by features
2. build a user profile from liked items
3. score unseen items by similarity

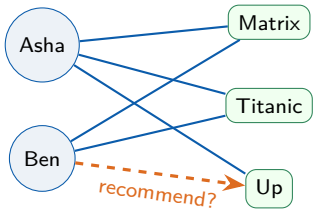
	Action	Sci-Fi	Romance
Matrix	1	1	0
Titanic	0	0	1
Avengers	1	1	0
Alien	1	1	0



Type 3: Collaborative filtering (the focus of Day 1)

Idea. Use the patterns of interaction of many users. No item features needed.

- *Users who agreed in the past will likely agree again*
- *Items liked by the same people are similar*



Asha and Ben liked the same movies;
Asha also liked Up ⇒ suggest Up to Ben

Two families (next sessions)

Neighbourhood methods: user-based, item-based (Lab 2)
Latent-factor methods: matrix factorization (Lab 3)

Type 4: Hybrid recommenders, and how the types compare

	Popularity	**Content-based**	**Collaborative**	**Hybrid**
Needs	interaction counts	item features	interaction matrix	both
Personalised	no	yes	yes	yes
New user	works	needs some history	fails	falls back to content
New item	fails	works	fails	works via content
Serendipity	low	low	high	medium–high
Main risk	rich-get-richer	filter bubble	sparsity, cold start	complexity

Hybrid designs: weighted blend | switching (e.g. popularity when cold) | features inside a CF model | cascade (retrieve, then re-rank)

What counts as feedback? Explicit vs implicit

Explicit

The user *tells* us:

- star ratings (1–5)
- thumbs up/down

\+ clear preference signal, negatives exist

– scarce, users self-select, rating habits differ

Implicit

We *observe* behaviour:

- click, view, watch time, purchase, add to cart, skip

\+ abundant, no extra effort

– noisy, no explicit negatives, confounded by what was shown

Remember

A rating of 4 says *what the user thinks*. A click only says *the user interacted*, maybe by accident.

Day 1 uses **explicit ratings** (MovieLens); implicit models (weighted ALS, BPR) come later.

The user–item rating matrix

$$R = \begin{bmatrix} 5 & 4 & ? & 1 \\ 4 & ? & 5 & ? \\ ? & 2 & 4 & 5 \end{bmatrix} \begin{matrix} \text{User 1} \\ \text{User 2} \\ \text{User 3} \end{matrix}$$

Movie A Movie B Movie C Movie D

- **rows** = users **columns** = items
- **values** = ratings ? = unknown preference
- $R \in (\mathbb{R} \cup \{?\})^{m \times n}$

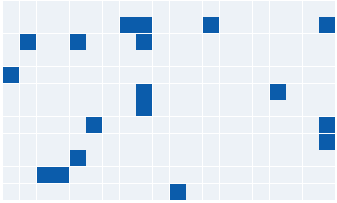
Reading the matrix

User 1 loves A, likes B, hates D.
User 3 likes C and D, dislikes B.
What would User 2 think of B? Of D?

Sparsity: most of the matrix is empty

Dataset	users	items	density
MovieLens latest-small	610	9 724	1.7 %
MovieLens 100K	943	1 682	6.3 %
Netflix Prize	480 189	17 770	1.2 %

Density = observed / $(m \times n)$.
MovieLens small: $100\,836 / 5\,931\,640 \approx 1.7\%$



users $\times$ items: blue = observed rating

Consequence

Two users rarely rate the same items $\Rightarrow$ similarity is unreliable.

Challenges you will meet today

Challenge	What goes wrong	Where we meet it
Sparsity	few co-rated items, unreliable similarity	Lab 1 (matrix), Lab 2
Cold start	new user / new item has no history	Lab 1 §6, Lab 2 §6
Popularity bias	popular items dominate the lists	Lab 1 §5, Lab 3 §9
User / item bias	generous raters, "good" movies inflate scores	Lab 1, SVD biases
Scalability	m^2 or n^2 similarities	Lab 2, MF
Evaluation	RMSE $\neq$ ranking quality; missing-not-at-random	Lab 3
Beyond accuracy	diversity, novelty, fairness, filter bubbles, privacy	discussion

How do we know a recommender is good?

Offline (today)

Hide some known ratings, predict them, compare.

Accuracy: RMSE, MAE
Ranking: Precision@N, Recall@N, NDCG
Beyond accuracy: coverage, diversity, novelty

Online (in production)

A/B tests with real users: clicks, watch time, retention, revenue.

Golden rules

1. Train / test split by data, never by hope
2. Always compare against a baseline
3. Tune on validation, report on test once

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{(u,i)}(r_{ui} - \hat{r}_{ui})^2} \qquad \mathrm{MAE} = \frac{1}{N}\sum_{(u,i)}|r_{ui} - \hat{r}_{ui}|$$

The Day 1 pipeline: keep this in view all day



Stage	Where	Question answered
Raw data, EDA	Lab 1	Who are my users and items? What does the data look like?
Similarity, CF	Theory 2, Lab 2	Who is similar to whom? Can neighbours fill the gaps?
Matrix factorization	Theory 3, Lab 3	Can hidden factors explain preferences?
Prediction, Top-N	Labs 2–3	What will user u think of item i? Which N items to show?
Evaluation	all labs	How good is it, compared with what?

Bridge to Lab 1: what to look for in the data

Dataset: MovieLens latest-small

- `ratings.csv`: `userId`, `movieId`, `rating`, `timestamp`
- `movies.csv`: `movieId`, `title`, `genres`
- 100 836 ratings, 610 users, 9 724 movies

Questions for the lab

1. Missing values? duplicates? rating scale?
2. How active are users? how popular are movies?
3. How sparse is the matrix?
4. What is the baseline error?

Get ready

Open `Lab1_EDA.ipynb`
Check `import pandas, sklearn, surprise`
Data in `data/ml-latest-small/`

Predict before you plot

Median ratings per user: 20, 70 or 500?
% of movies with one rating: 5, 35 or 70?

Summary and check questions

- A recommender predicts which **unseen items** a user will find relevant
- Four families: **popularity, content-based, collaborative, hybrid**
- Feedback is **explicit** (ratings) or **implicit** (behaviour)
- Data = a huge, sparse user–item matrix; the job is to *fill the question marks*
- Two tasks: **rating prediction** (RMSE) and **Top-N** (Precision@N)

Check yourself

1. What problem does a recommender solve?
2. Give one pro and one con of content-based filtering.
3. Why is the rating matrix sparse?
4. Explicit or implicit: a "skip" on Spotify?

References and further reading I

- F. Ricci, L. Rokach, B. Shapira (eds.), *Recommender Systems Handbook*, Springer, 3rd ed. 2022.
- C. Aggarwal, *Recommender Systems: The Textbook*, Springer, 2016.
- P. Resnick et al., GroupLens: an open architecture for collaborative filtering of netnews, *CSCW*, 1994.
- G. Linden, B. Smith, J. York, Amazon.com recommendations: item-to-item collaborative filtering, *IEEE Internet Computing*, 2003.
- Y. Koren, R. Bell, C. Volinsky, Matrix factorization techniques for recommender systems, *IEEE Computer*, 2009.
- C. Gomez-Uribe, N. Hunt, The Netflix recommender system: algorithms, business value, and innovation, *ACM TMIS*, 2015.
- F. M. Harper, J. Konstan, The MovieLens datasets: history and context, *ACM TiiS*, 2015.
- C. Anderson, The long tail, *Wired*, 2004.
- N. Hug, Surprise: a Python library for recommender systems, *JOSS*, 2020.